

Chrome Session Buddy

User Guide

chrome_report (latest: chrome_report_v1.1.py)

— generate HTML (and JSON) reports of your open Chrome tabs, organised by tab group

chrome_report_edit

— a local web app to edit those reports (rename windows, delete / insert entries, save under a new name)

Self-contained tools — Python standard library only, no dependencies to install.

Contents

- 1 Overview & workflow
- 2 chrome_report (latest version: chrome_report_v1.1.py)
 - 2.1 What it does
 - 2.2 Usage & options
 - 2.3 Output files
 - 2.4 How it works
 - 2.5 Notes & troubleshooting
- 3 chrome_report_edit (chrome_report_edit.py)
 - 3.1 What it does
 - 3.2 Startup & options
 - 3.3 Editing capabilities
 - 3.4 Saving
 - 3.5 Safety & notes
- 4 Typical workflow
- 5 Troubleshooting & FAQ

1 Overview & workflow

These two tools work together as a small “session buddy”: `chrome_report` turns your currently open Chrome windows and tabs into a self-contained HTML report (and optionally JSON), and `chrome_report_edit` lets you polish that report by renaming windows, editing or deleting entries, inserting new entries, and saving the result under a name you choose.

Both tools are single Python files using only the standard library. They run on Windows, macOS and Linux, require no *pip install*, and never talk to any external service. They read Chrome’s own session data directly from disk, so Chrome itself does not need to be closed — it can keep running while you generate a report.

Typical flow

1. `python chrome_report_v1.1.py --json --outdir reports` → create the report(s).
2. `python chrome_report_edit.py --dir reports` → open the editor in your browser.
3. Edit, then **Save as** a new name, e.g. `final_2026.html`.

2 `chrome_report` (latest version: `chrome_report_v1.1.py`)

2.1 What it does

`chrome_report_v1.1.py` reads one Chrome profile — the **Default** profile — and produces a report of every window it finds. For each window it lists every tab in three columns: **Group** (the tab group the tab belongs to, left blank when ungrouped), **Tab** (the page title, clickable), and **URL** (clickable). Tab groups appear as colour-coded chips that match the colours you see in Chrome.

Windows are simply named **Window 1**, **Window 2**, ... **Window N** (numbered per report). Each window block shows how many tabs (and groups) it contains; the active window and the active tab are marked with a star.

2.2 Usage & options

```
python chrome_report_v1.1.py [--data-dir PATH] [--outdir PATH] [--json]
```

Option	Meaning
<code>(none)</code>	Uses the Default profile, auto-detects the Chrome data directory, and writes the HTML report into the current folder.
<code>--data-dir PATH</code>	Chrome profile data directory instead of auto-detect. Auto-detect looks here: • Windows — <code>%LOCALAPPDATA%\Google\Chrome\User Data</code> • macOS — <code>~/Library/Application Support/Google/Chrome</code> • Linux — <code>~/.config/google-chrome</code>
<code>--outdir PATH</code>	Directory for the output file(s). Created if missing. Default: current directory.
<code>--json</code>	Additionally write the parsed data as a JSON file (see 2.3).

2.3 Output files

Written into `--outdir`:

File	Contents
chrome_tabs_Default_v1.1.html	Self-contained report: inline CSS, all styling embedded, links open in your default browser. Safe to share or keep as a personal archive.
chrome_tabs_Default_v1.1.json	(only with <code>--json</code>) Machine-readable data: <code>profile</code> , <code>files</code> (source session files), <code>groups</code> , and <code>windows[]</code> , each window with <code>id</code> , <code>name</code> , <code>active</code> and <code>tabs[]</code> (url, title, group, color, active).

2.4 How it works

Chrome continuously saves its “session” to binary files inside `<data dir>\<profile>\Sessions\` (files such as `Session_...` and `Tabs_...`). The script parses this SNSS format directly — including tab-group membership and group names/colours — so no Chrome window, extension or automation API is required. Files that are mid-write are retried a few times; anything still locked is skipped with a warning (see 2.5). The report contains the most recent complete snapshot that was readable.

The script reads only the **Default** profile. Earlier versions (`chrome_report_v1.py`, `chrome_report.py`) supported other profiles; this latest version intentionally focuses on Default so its output filenames are fixed and predictable (no `--profile` flag).

2.5 Notes & troubleshooting

- **Chrome can keep running** while you generate the report; nothing is closed or modified. Chrome only needs its session files on disk (it writes them constantly while running).
- `warning: skipped (... , in use by Chrome)` — the newest session file was being written at that exact moment. The report still contains the previous complete snapshot; wait a few seconds and re-run for the very latest state.
- Duplicate group names across Chrome restarts are counted **once** (groups are identified by name and value), so the group totals stay accurate.
- If no windows/tabs are found, double-check you are running the script on the machine where that Chrome profile lives (session files are local to it).

3 chrome_report_edit (chrome_report_edit.py)

3.1 What it does

`chrome_report_edit.py` is a small local web app for editing a generated HTML report. It starts a **localhost-only** web server, opens the editor in your browser, and lets you make changes that are written back to disk when you save. The report is shown exactly as it will look (inside a preview pane that keeps the report’s own styling), and statistics — windows · tabs · groups — update as you edit.

3.2 Startup & options

```
python chrome_report_edit.py [--dir PATH] [--port 8765] [--host 127.0.0.1] [--no-browser]
```

Option	Meaning
(none)	Serves the reports in the script's own folder and opens the browser automatically.
--dir PATH	Directory whose *.html files the editor manages (and where saved files are written). Default: the script's folder.
--port N	Port for the local server. Default 8765.
--host ADDR	Bind address. Default 127.0.0.1 (loopback only).
--no-browser	Do not auto-open the browser; the URL is printed instead.

After starting, a browser tab opens at <http://127.0.0.1:8765/>. To edit a report you either (a) pick it from the **dropdown** at the top and click **Load**, or (b) click **Open...** to load any *.html file from anywhere. Press **Ctrl+C** in the terminal to stop the server.

3.3 Editing capabilities

1. **Change the window names.** Click any "Window N" heading and type a new name. The new name is used in the window header and in the insert-dialog's window list.
2. **Delete entries.** Each table row has a × button; clicking it removes that tab. The window's tab/group count and the report's header statistics are recomputed immediately. (Tip: **Ctrl+Backspace** while a row is focused also deletes it.)
3. **Insert a new entry.** Click **Add tab** (in a window header, appends at the end) or the + button on a specific row (inserts after that row). A dialog then asks for:
 - **Window** — which window receives the tab (drop-down, defaults to the current one);
 - **Position** — at the top, after a specific row, or at the end;
 - **Group** (optional) — pick from existing group names (auto-suggested, colour reused) or type a new one;
 - **Tab name** and **URL**.

The new row is added using the report's exact markup, so it is indistinguishable from generated rows in the saved file.

4. **Save the modified report under a user-defined name.** Type the file name in the toolbar (or keep the current one) and press **Save** (or Enter). The file is written into the managed directory and the dropdown refreshes to list it.

In place edits on existing rows (extra): besides deleting and inserting, you can click any row's group chip, tab title or URL and edit the text directly. Editing a URL also updates the link's href and tooltip. Clearing a group name ungroups the tab; typing one adds it with a default colour.

3.4 Saving

- Nothing is written to disk until you press **Save**; loading or editing alone changes nothing on disk.
- The name must be a simple `*.html` file name (no paths, no slashes). The server validates this and always writes inside the managed directory — traversal attempts (`..\`, absolute paths) are rejected.
- On success the status bar shows the full path and byte count of the saved file, and it appears in the file dropdown. The browser may prompt before closing the tab if you have unsaved edits.

3.5 Safety & notes

- The server binds to `127.0.0.1` by default and is intended for local use only.
- Only `*.html` files are listed/loaded/saved; query strings and names are validated, so the tool cannot read or write anything outside `--dir`.
- The preview pane is sandboxed (scripts allowed, navigation blocked), so clicking a link in the report will not navigate the editor away.
- Loading an edited report back in the editor works the same way as the original — you can edit your saved revisions repeatedly.

4 Typical workflow

```
# 1) Generate the report for the Default profile, with JSON, into ./reports
python chrome_report_v1.1.py --json --outdir reports

# 2) Open the editor pointed at that folder (browser opens automatically)
python chrome_report_edit.py --dir reports

# 3) In the browser: Load chrome_tabs_Default_v1.1.html
#   - click "Window 3" heading, type "Hi-C"
#   - remove a few stale rows with x
#   - use + next to a row to insert a new tab (ChIP / bioRxiv / URL)
#   - after each change the tab/group counts update

# 4) Type "final_2026.html" in the name box and press Save
#   -> reports/final_2026.html is written and listed

# 5) Open the saved file (or keep editing it in the editor)
start reports\final_2026.html
```

5 Troubleshooting & FAQ

Symptom / question	Answer
Script prints "warning: skipped (... in use by Chrome)"	Chrome was writing its newest session file at that moment. The report still shows the previous complete snapshot. Wait a few seconds and re-run <code>chrome_report_v1.1.py</code> .

"Default profile not found"	The script could not find a <code>Preferences</code> file under <code><data dir>\Default</code> . Point <code>--data-dir</code> at the right "User Data" directory, or run on the machine that uses this Chrome profile.
No reports in the editor dropdown	The dropdown lists <code>*.html</code> files in <code>--dir</code> only. Start the editor with the folder that contains your report (e.g. <code>--dir reports</code>), or use Open... to load any file.
"Address already in use" when starting the editor	Another process (or an earlier editor instance) uses port 8765. Run <code>python chrome_report_edit.py --port 9000</code> (any free port).
Can I edit reports generated for other profiles?	Yes — the editor works on any report in this HTML format. Only the <i>generator</i> (<code>chrome_report_v1.1.py</code>) is restricted to the Default profile. Older generators (<code>chrome_report_v1.py</code> , <code>chrome_report.py</code>) can produce reports for other profiles, which the editor can then load and edit.
Are the tools safe to run? Do they install anything?	Yes — standard library only, nothing is installed, no network access required, nothing is modified on disk except via an explicit "Save" in the editor.
Must Chrome be closed to generate a report?	No. Reports are read from the session files Chrome writes continuously while running. If a file is mid-write it is skipped with a warning and reported as a re-run hint.